«Python desde cero»

«Del primer print al eco radar»

Aprender a programar para comprender
señales, cálculos y vigilancia.

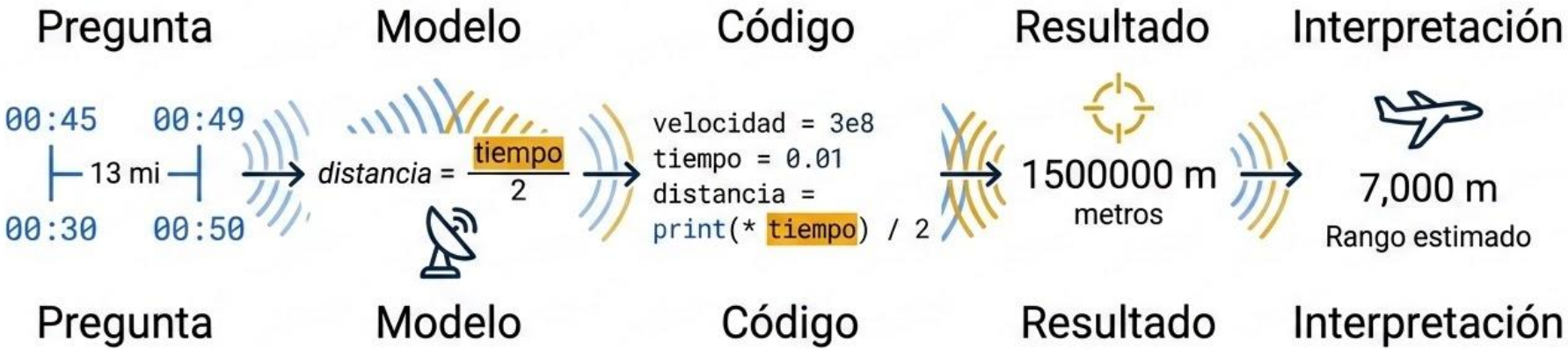
Marco Terán IQV01



```
distancia = velocidad * tiempo / 2
```

¿Por qué aprender Python?

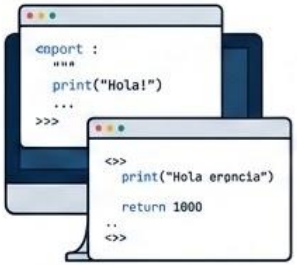
«De una pregunta técnica a una respuesta verificable.»



Python no reemplaza el razonamiento: permite comprobarlo.

«La ruta de aprendizaje»

«Primero comprendemos la herramienta; después, el fenómeno.»



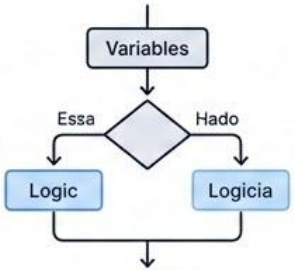
1 · Python

Demonstraja el basic language cadi with no prior in programming.



2 · Notebook y Colab

Interactivo notebook o 'Python (similar or Colab con horenarti) y Colab.



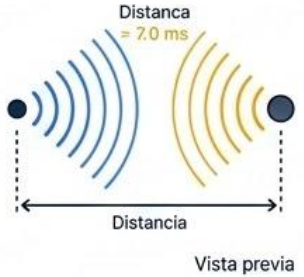
3 · Programar

Concept structurar programaminas en decision H' y lógic.



4 · Laboratorio 01

Leanna: see la apioratorio et iercises amienomy de Laboratorio 01.



5 · Hacia el radar

Application del radar system y es oocientos treoria using Python.

Python × Vigilancia · IQV01 | 03/26

¿Qué es Python?

Un lenguaje para escribir instrucciones ejecutables.

Escribir 

```
print("Hola, vigilancia")
```

Ejecutar → **Observar**

```
Hola, vigilancia
```

print() muestra información; las comillas delimitan texto.

Python traduce sus instrucciones en acciones legibles por la computadora.

Python: historia, evolución y popularidad

De 1991 a uno de los lenguajes más utilizados.



58 %



58 % de los encuestados utilizó Python

8.184 de 14.099 participantes · Stack Overflow 2026

Stack Overflow Developer Survey 2026

La cifra corresponde a una encuesta, no al total mundial de usuarios.

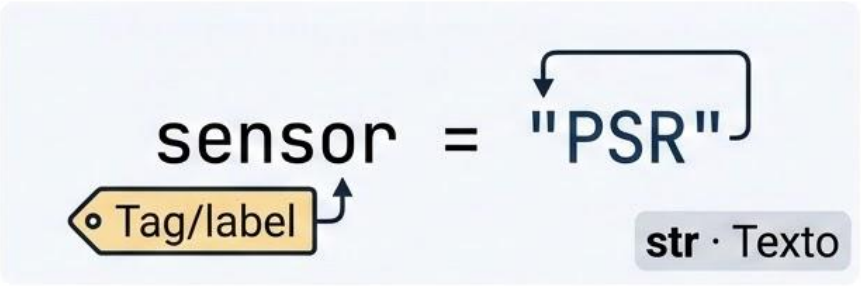
Alto nivel · Interpretado · Propósito general · Tipado dinámico

Variables y tipos de datos

Una variable tiene un nombre; su valor tiene un tipo.





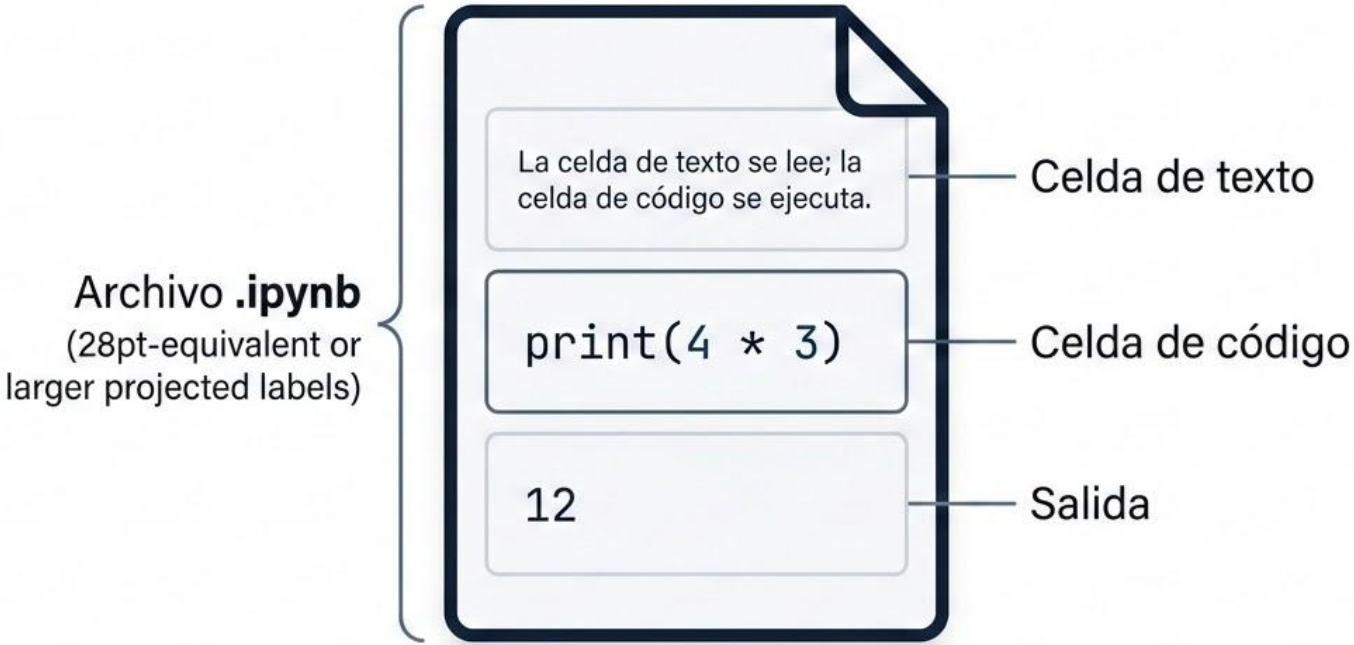




El signo = asigna un valor.

¿Qué es un notebook?

Un documento con explicación, código y resultados.



«Python, Jupyter, Colab y Anaconda»

Cuatro herramientas relacionadas; cuatro funciones distintas.

Python · lenguaje de programación

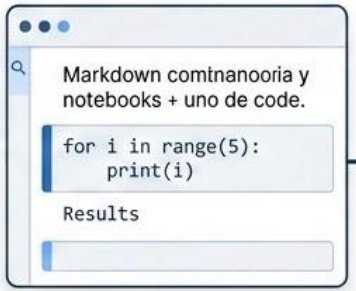
https://www.python.org/community/logos/

```
for i in range(5):
    print(i)
...
```

Concisie, real readable Python, conto se usa a da amoadem, readae navy.

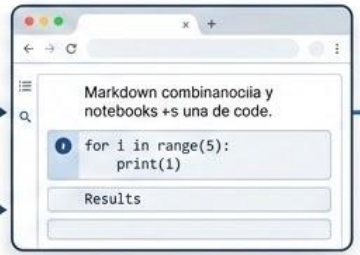
Jupyter · ecosistema de notebooks

https://github.com/jupyter/design



Colab · ejecución en el navegador

https://colab.research.google.com/



Ejecuta notebooks en la nube de Google, sin configuración local.

Anaconda · distribución y entornos locales

https://www.anaconda.com/



Gestiona paquetes de ciencia de datos y entornos aislados en tu computadora.

Para comenzar hoy: Colab. Anaconda es opcional.

«Entrar a Google Colab»

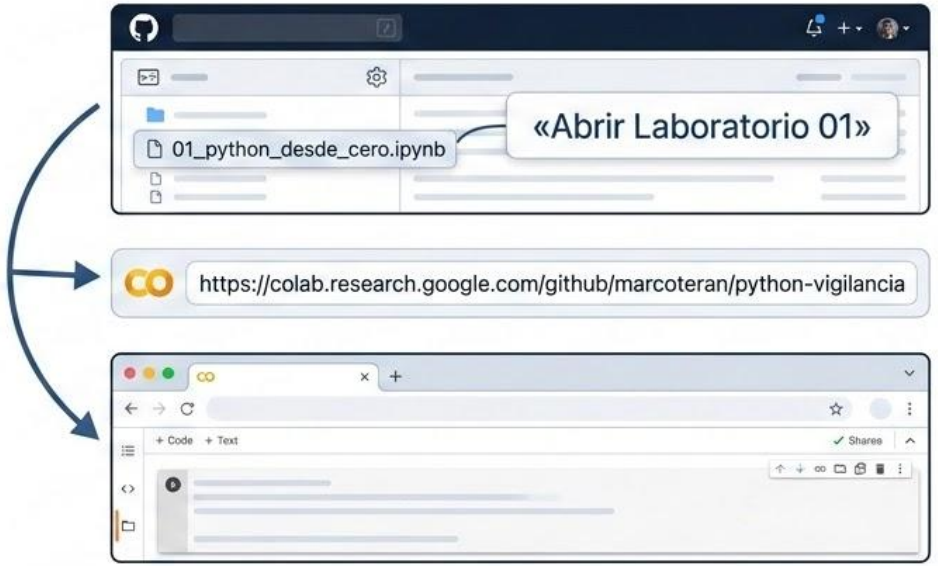
«Laboratorio 01, sin instalar Python.»

1 · Abre el enlace del curso

2 · Inicia sesión si se solicita

3 · Comprueba el nombre del notebook

4 · Abre el laboratorio



`01_python_desde_cero.ipynb`

También puedes abrirlo desde el README en GitHub.

«Tu primera sesión en Colab»

Google Colab

«Guardar una copia → conectar → ejecutar → comprobar.»



«Alternativa local: Anaconda + JupyterLab»

«Un notebook también puede ejecutarse en tu computador.»



1 · Descarga Anaconda Distribution

2 · Instala y abre Anaconda Navigator

3 · Inicia JupyterLab

4 · Abre tu archivo **.ipynb**

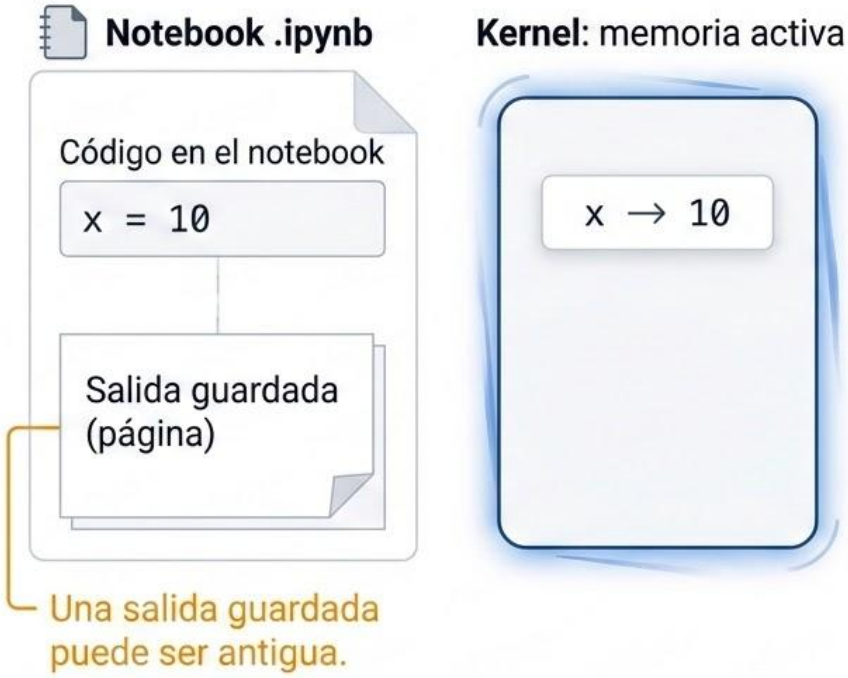
5 · Selecciona el kernel de Python

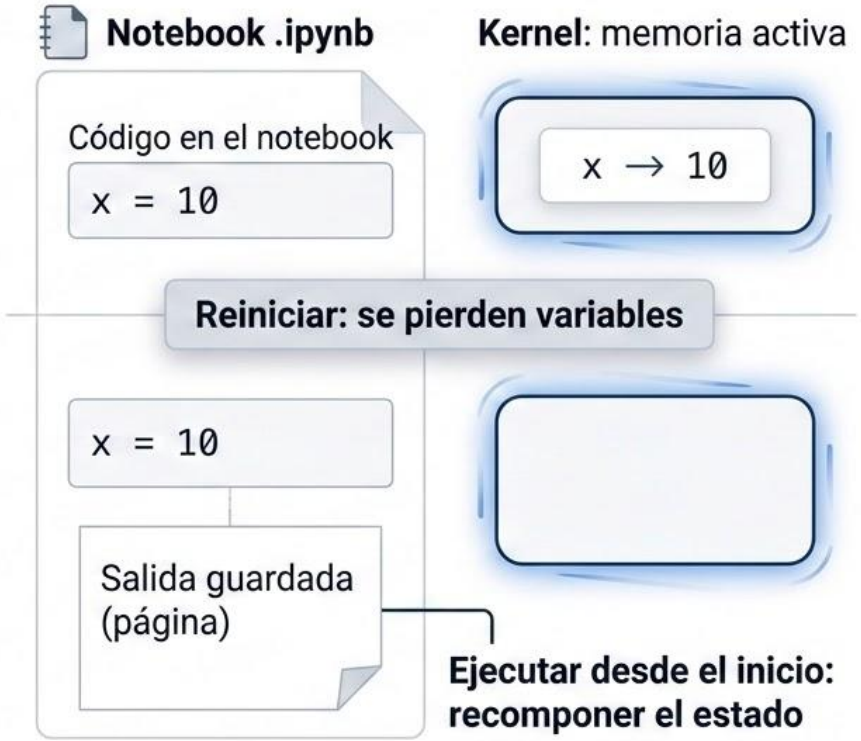
Opcional: para Laboratorio 01 basta Colab.

Python × Vigilancia · IQV01 | 09/26

«El documento no es la memoria»

«El código se guarda; las variables viven en la sesión.»





«Variables, operaciones y unidades»

Un número no basta si no sabes qué representa.

Código en el notebook

```
minutos = 84
horas = minutos / 60
print(f"{horas:.1f} h")
```

Salida (output)

```
1.4 h
```

FLUJO DE CONVERSIÓN



= asigna | == compara | ** eleva a potencia

= asigna | / divide | == compara | ** eleva a potencia

El resultado necesita unidad e interpretación.

«No todos los datos son iguales»

El tipo de dato cambia lo que Python puede hacer.

```
muestras = 12
```



Entero · int

```
frecuencia_mhz = 1090.0
```



Decimal · float

```
sistema = "ADS-B"
```





Texto · str

```
activo = True
```



Lógico · bool

«Listas: valores en orden»

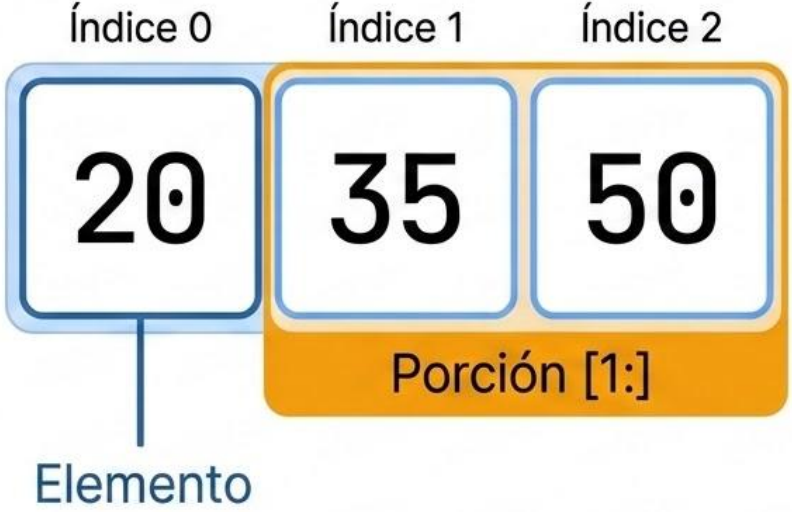
En Python, la primera posición es 0.

Código en el notebook

```
tiempos = [20, 35, 50]
print(tiempos[0])
print(tiempos[1:])
```

Salida (output)

```
20
[35, 50]
```



«Condiciones: elegir cuándo actuar»

«if ejecuta un bloque si la condición es verdadera.»

```
tiempo_min = 75

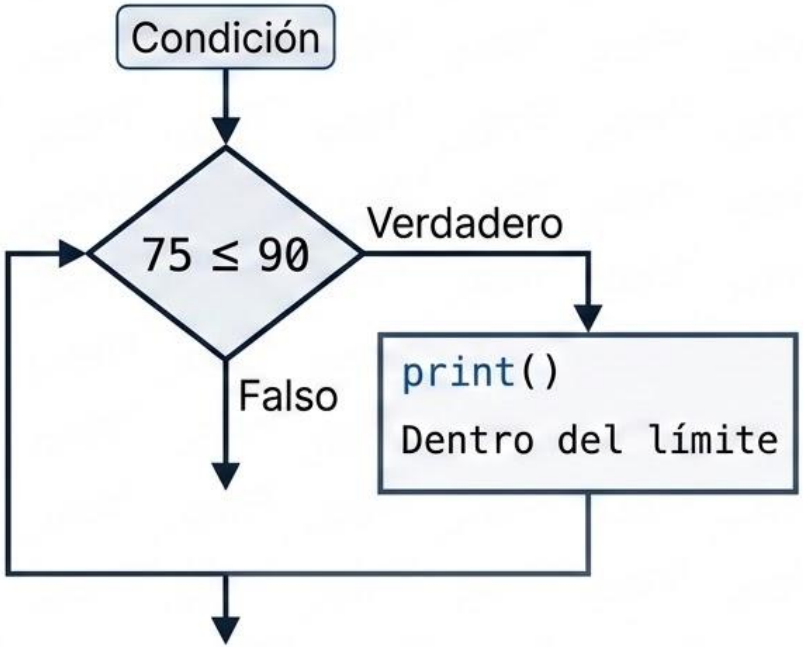
if tiempo_min <= 90:
    print("Dentro del límite")
```

Bloque indentado

Cuatro espacios

Salida (output)

```
Dentro del límite
```



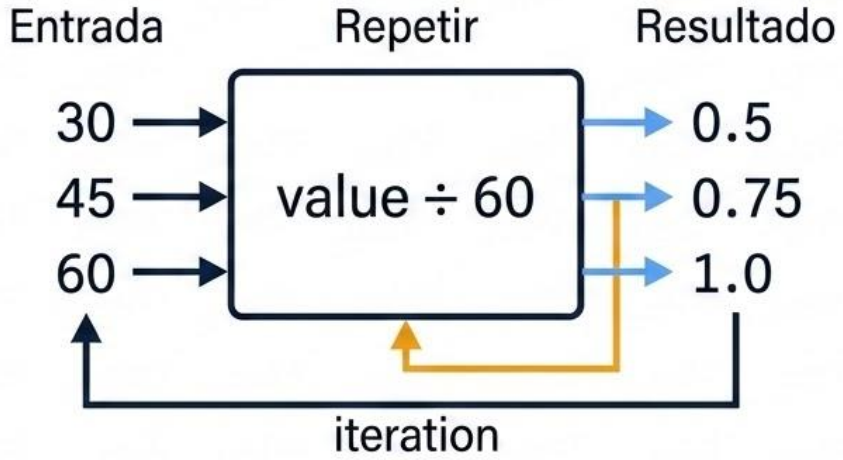
«Ciclos: repetir sin copiar código»

«**for** recorre los elementos uno a uno.

```
for m in [30, 45, 60]:
    print(m / 60)
```

stdout

```
0.5
0.75
1.0
```



Tiny optional: Forma abreviada: **comprensión de lista**

```
[m / 60 for m in [30, 45, 60]]
```

«Funciones: reutilizar un cálculo»

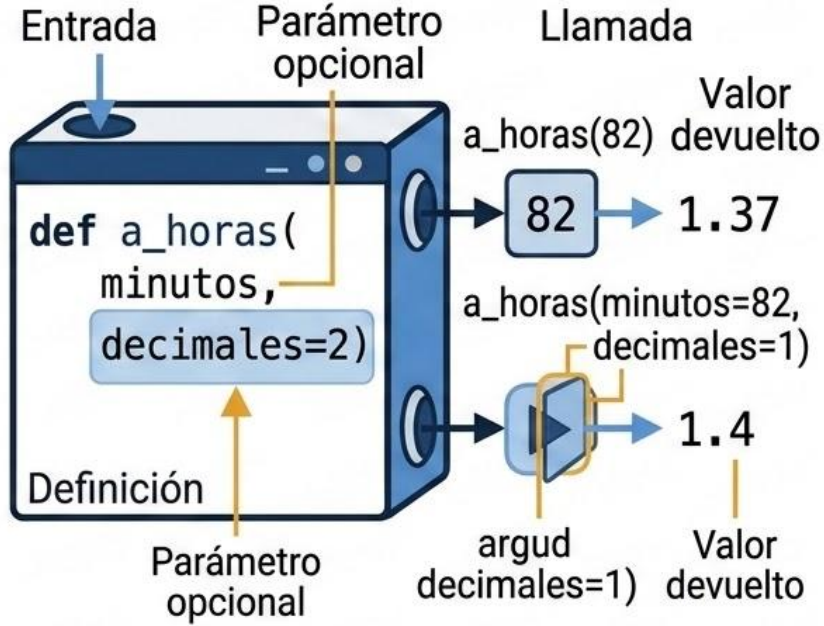
«**Definir** una función no equivale a ejecutarla.»

```
def a_horas(minutos, decimales=2):
    return round(minutos / 60,
        decimales)

a_horas(82)
a_horas(minutos=82, decimales=1)
```

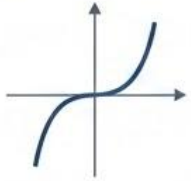
Salida (output)

```
Primera llamada -> 1.37
Segunda llamada -> 1.4
```



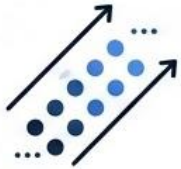
«Bibliotecas: herramientas listas para usar»

Importar no es lo mismo que instalar.



math · funciones matemáticas



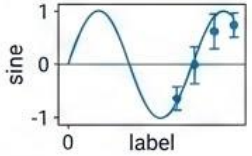


NumPy · arreglos y cálculo

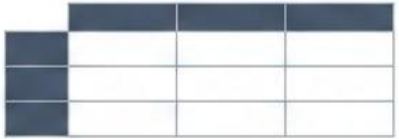
`[ 1 5 4 3 2 1 0 ,...,[10]`

`import numpy` → `np` ← Alias

Matplotlib · gráficas



pandas · tablas (ampliación)



```
import numpy as np
```

Disponible en el **entorno** ⟶ **Importar** ⟶ Usar

Python × Vigilancia · IQV01 | 17/26

«NumPy: operar con muchos valores»

Un arreglo aplica cálculos a varios elementos.




```
import numpy as np
t = np.array([30, 60, 90])
print(t / 60)
```

```
[0.5 1. 1.5]
```

`np.linspace`: puntos espaciados
`np.arange`: secuencia
`np.log10`: logaritmo decimal

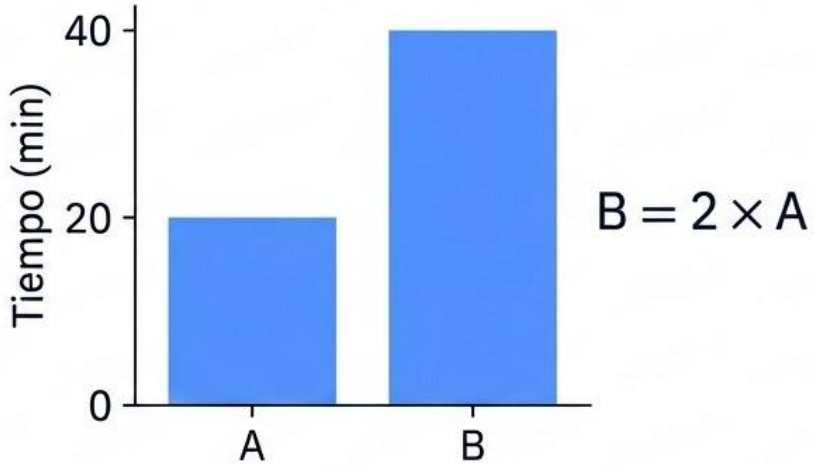
«Matplotlib: ver lo que calculamos»

«Una gráfica debe responder una pregunta.»

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4))
plt.bar(['A', 'B'], [20, 40], color='#4285F4')
plt.title('Comparación Simple')
plt.ylabel('Tiempo (min)')
plt.ylim(0, 50) # Ensure scale is comparable
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()
```



Código Limpio, Ejecución Rápida

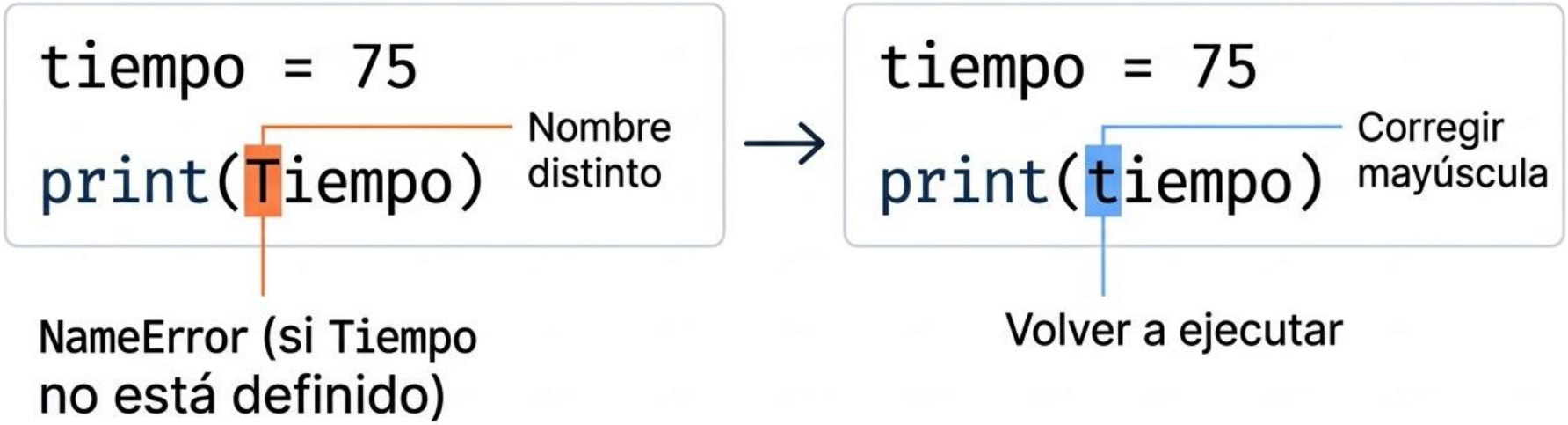
Lee los ejes

Comprueba las unidades

Interpreta la relación

«Los errores también enseñan»

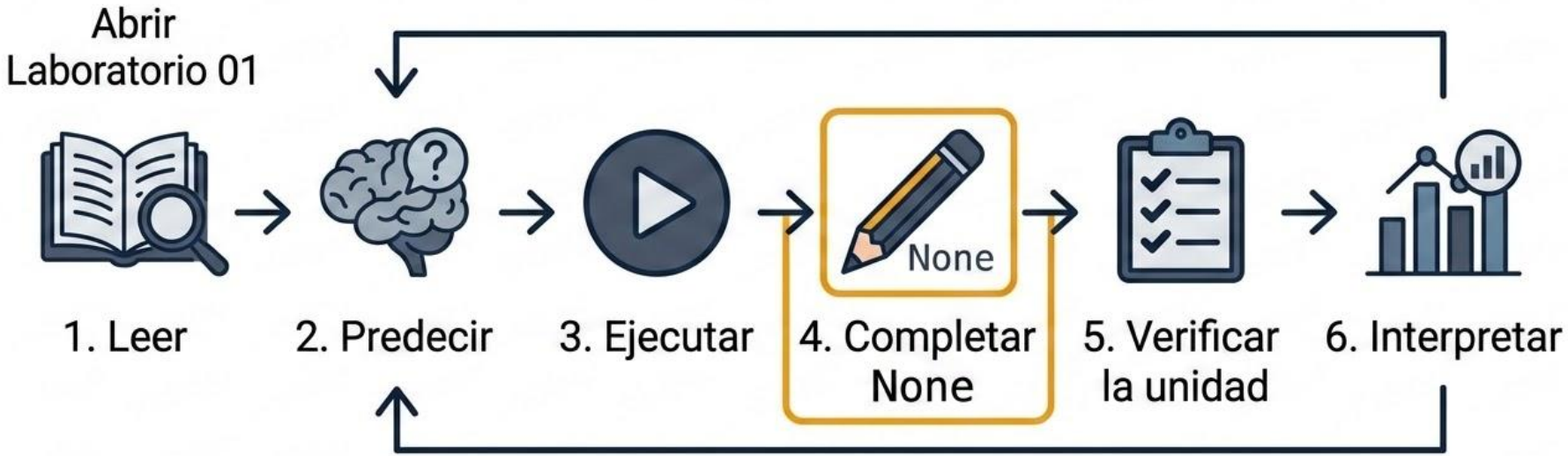
«Leer el error → encontrar la causa → corregir.»



Python distingue mayúsculas y minúsculas.

«Cómo trabajar el Laboratorio 01»

«Leer, predecir, ejecutar, completar e interpretar.»

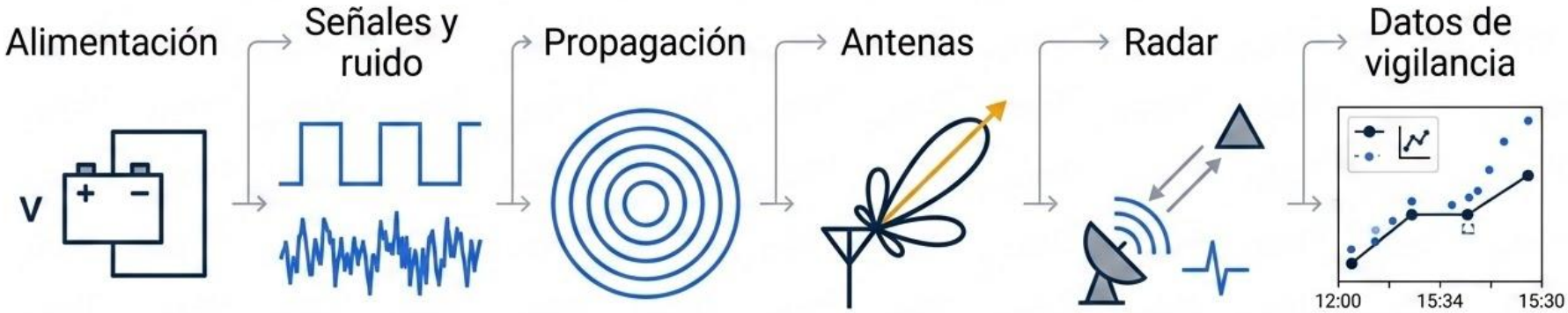


E01–E04	Misión M01–M04	Puente R01

Si falla, revisa el orden de las celdas.

«Laboratorio 02: Python y vigilancia»

Del modelo matemático a su significado físico.



Modelo → Cálculo → Gráfica → Interpretación

«Del pulso al alcance radar»

«Si el eco tarda 100 µs, ¿qué distancia recorrió de ida?»



$$R = \frac{c \cdot t}{2} \rightarrow \text{Alcance} \approx 15\ \text{km}$$

$c \approx 3 \times 10^8\ \text{m/s}$

$t = 100\ \mu\text{s}$

REQUIRED CODE

```
c = 3e8
t = 100e-6
print(c * t / 2 / 1000, "km")
```

REQUIRED RESULT OUTPUT

```
15.0 km
```

«Lo esencial para comenzar»

Entender → probar → explicar.

Python ejecuta instrucciones

Un notebook combina explicación, código y salidas



Ahora: Laboratorio 01

Colab permite practicar **en el navegador**

Anaconda y JupyterLab son una alternativa **local**

Un resultado necesita unidad e interp**retación**

«Preguntas»

¿Qué parte de escribir, ejecutar o interpretar te genera dudas?»

Código: Entender la sintaxis y depurar errores.



Notebook: Gestionar celdas de texto y código.



Colab / Jupyter: Configurar y usar el entorno.



«Gracias»

«Ahora, abramos el Laboratorio 01.»

